MCMP117 · WEEK 12

Fundamentals of Operating Systems

Week 12 · Text Processing, Redirection & Pipes

Small programs, connected by the kernel, answer real questions about real data.

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 11 in Four Ideas

Users and groups

id, whoami, groups; every user and group has a number (UID, GID), listed in /etc/passwd and /etc/group.

The mode string

-rwxr-x--- : file type, then rwx for owner, group and others. On a directory, x means "may enter".

chmod

Symbolic (u+x, go-w) or octal with r = 4, w = 2, x = 1: 754 = rwx r-x r--.

Who may change it

umask sets defaults; chown needs root; sudo runs one command as root — use it carefully.

Today the files we protected last week become data: we search, cut, sort and count their contents, and we connect programs to each other.

Today's Agenda

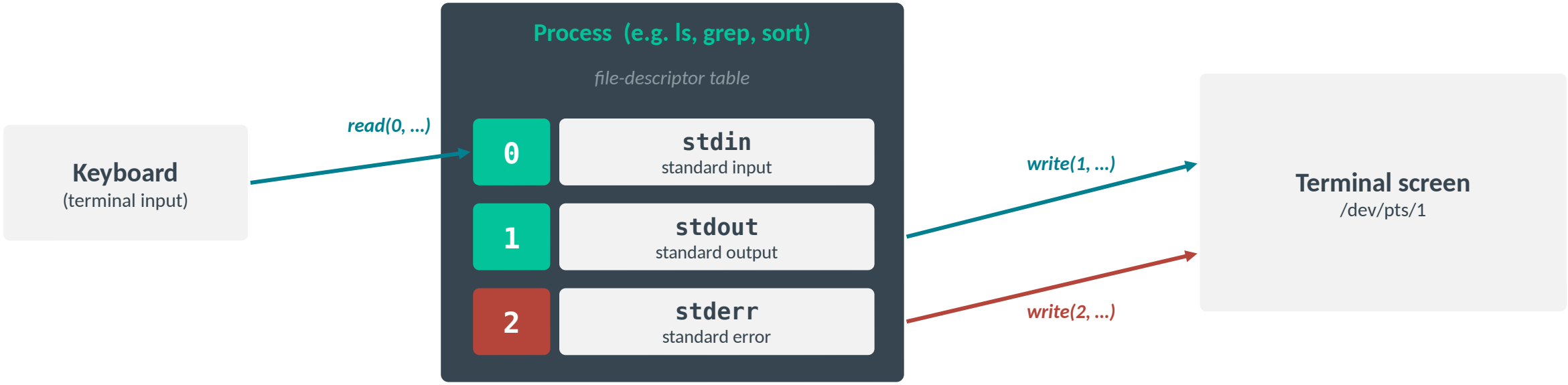
1. **Standard streams** stdin, stdout, stderr = file descriptors 0, 1, 2 (Weeks 2–3: write(1, ...)).
2. **Redirection** > >> < 2> 2>&1 &> /dev/null — and how the shell does it with dup2().
3. **Pipes** cmd1 | cmd2: two processes connected by a kernel buffer.
4. **Filters** grep (with simple regular expressions), sort, uniq, cut, wc, tr.
5. **More tools** head and tail in pipelines, tee, find, xargs.
6. **Real analyses + lab** Status codes, top pages, errors per hour, grades — one line each.

PART 1

Standard Streams & Redirection

Every process starts life with three open files.

Three Streams, Three Small Numbers



0 · stdin

Where the program reads its input. Default: the keyboard.

1 · stdout

Normal results. Default: the terminal screen.

2 · stderr

Error messages, on a separate channel — so they still reach you when the results go to a file.

You Have Seen This Before: write(1, …)

```
student@okan-lab:~/lab$ strace -o t1.txt -e trace=write ls data nosuch
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat t1.txt
write(2, "ls: ", 4)                      = 4
write(2, "cannot access 'nosuch'", 22)  = 22
write(2, ": No such file or directory", 27) = 27
write(2, "\n", 1)                        = 1
write(1, "data:\n", 6)                   = 6
write(1, "cities.txt  numbers.txt  student"..., 38) = 38
+++ exited with 2 +++
```

What strace shows

- The error message goes out with write(2, …): fd 2, stderr.
- The listing goes out with write(1, …): fd 1, stdout.
- ls never asks where 1 and 2 lead: that was decided before it started.

Week 2: printf() ends as write(1, "Hello…", n). Week 3: a system call is a trap into the kernel, and fd 1 means standard output. Today: if the shell makes fd 1 point to a file before ls starts, every write(1, …) lands in that file. That is redirection.

Seeing the Descriptors: ls -l /proc/self/fd

```
student@okan-lab:~/lab$ ls -l /proc/self/fd
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15829/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd > fd.txt
student@okan-lab:~/lab$ cat fd.txt
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
l-wx------ 1 student student 64 Oct 12 08:48 1 -> /home/student/lab/fd.txt
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15832/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd 2> /dev/null
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
l-wx------ 1 student student 64 Oct 12 08:48 2 -> /dev/null
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15841/fd
```

What to observe

- /proc/self is the process that is looking — here ls itself (Week 3: /proc).
- Normally 0, 1, 2 all point to the terminal /dev/pts/1.
- With > fd.txt, fd 1 points to the file: ls wrote its own listing into it.
- With 2> /dev/null, fd 2 points to the "black hole" device.
- fd 3 is the directory ls is reading.

Output Redirection: > Overwrites, >> Appends

```
student@okan-lab:~/lab$ echo "first line" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
student@okan-lab:~/lab$ echo "second line" >> notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
second line
student@okan-lab:~/lab$ echo "oops" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
oops
```

The rules

- **cmd > file** creates the file, or empties it first (truncate), then writes stdout into it.
- **cmd >> file** creates it if needed, then adds at the end.
- Nothing appears on the screen: stdout went to the file.

Careful

> empties the file before the command even starts. One wrong > destroys a file — there is no undo and no recycle bin (Week 10).

Input Redirection: < Feeds a File into stdin

```
student@okan-lab:~/lab$ wc -l logs/access.log
300 logs/access.log
student@okan-lab:~/lab$ wc -l < logs/access.log
300
student@okan-lab:~/lab$ ls -l /proc/self/fd < docs/syllabus.txt
total 0
lr-x------ 1 student student 64 Oct 12 08:48 0 ->
/home/student/lab/docs/syllabus.txt
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15838/fd
```

Two ways to read a file

- **wc -l file:** wc opens the file itself and knows its name, so it prints it.
- **wc -l < file:** the shell opens it as fd 0; wc only sees a stream, so no name.

Most text tools follow the filter convention: read the files named as arguments, or stdin if there are none; write the result to stdout. That is why they can be chained with pipes — the main topic of today.

Redirecting Errors: 2>, 2>&1, &> and /dev/null

```
student@okan-lab:~/lab$ ls data nosuch > out.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ ls data nosuch 2> err.txt
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat err.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ ls data nosuch > all.txt 2>&1
student@okan-lab:~/lab$ cat all.txt
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt
numbers.txt
students.csv
student@okan-lab:~/lab$ ls data nosuch 2> /dev/null
data:
cities.txt  numbers.txt  students.csv
```

Syntax	Meaning
`> f  or  1> f`	stdout to f
`2> f`	stderr to f
`2>> f`	append stderr to f
`> f 2>&1`	both to f (fd 2 = copy of fd 1)
`&> f`	both to f (bash shorthand)
`2> /dev/null`	throw errors away

/dev/null is a device file: everything written to it disappears; reading from it gives end-of-file at once.

Order Matters: Redirections Run Left to Right

`ls data nosuch > all.txt 2>&1`

start	fd 1 → terminal	fd 2 → terminal
> all.txt	**fd 1 → all.txt**	fd 2 → terminal
2>&1	**fd 1 → all.txt**	**fd 2 → all.txt**

Both streams end up in all.txt.

`ls data nosuch 2>&1 > out2.txt`

start	fd 1 → terminal	fd 2 → terminal
2>&1	fd 1 → terminal	fd 2 → terminal
> out2.txt	**fd 1 → out2.txt**	fd 2 → terminal

The error still appears on the screen!

```
student@okan-lab:~/lab$ ls data nosuch 2>&1 > out2.txt
ls: cannot access 'nosuch': No such file or directory
# 2>&1 means: make fd 2 a copy of what fd 1 is NOW — not a permanent link to fd 1
```

How the Shell Does It: fork, open, dup2, exec

```
student@okan-lab:~/lab$ strace -f -o t2.txt -e trace=openat,dup2,execve bash -c "ls data > list.txt"
student@okan-lab:~/lab$ grep -E "list.txt|dup2|execve" t2.txt
15726 execve("/usr/bin/bash", ["bash", "-c", "ls data > list.txt"], …) = 0
15727 openat(AT_FDCWD, "list.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
15727 dup2(3, 1)                        = 1
15727 execve("/usr/bin/ls", ["ls", "data"], …) = 0
```

1 fork (clone)

bash creates a child: a copy of itself, with the same fds 0, 1, 2.

2 openat

The child opens list.txt for writing (O_TRUNC = empty it). New fd: 3.

3 dup2(3, 1)

fd 1 becomes a copy of fd 3: stdout now leads to list.txt.

4 execve ls

The child becomes ls. The fd table survives exec — ls writes to fd 1.

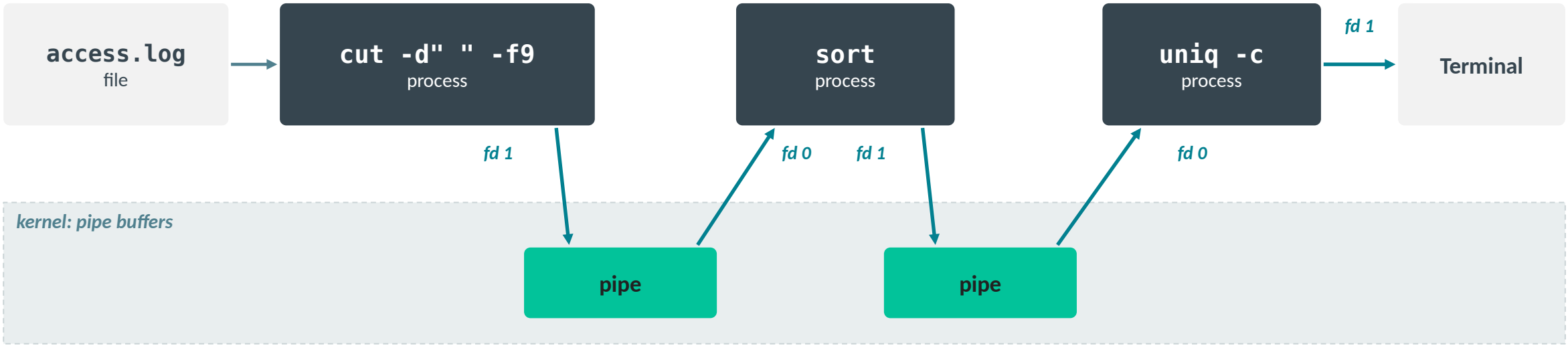
Week 3 asked: why are fork() and exec() two separate calls? Because between them the child can rearrange its file descriptors — and the program it then runs never knows. Linux implements fork() with the clone() system call.

PART 2

Pipes

The output of one process becomes the input of the next.

A Pipeline: Processes Connected by Pipes




```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c
    148 200
     20 301
     34 304
     17 403
     57 404
     24 500
```

- Three processes run at the same time; each pipe is a buffer inside the kernel.
- cut writes to fd 1 = the pipe; sort reads fd 0 = the pipe.
- stderr (fd 2) of every stage still goes to the terminal.

LINK TO WEEK 3

Building a Pipe: pipe2, clone, dup2, execve

```
student@okan-lab:~/lab$ strace -f -o t3.txt -e trace=pipe2,clone,dup2,execve bash
-c "grep ERROR logs/system.log | wc -l"
13
student@okan-lab:~/lab$ cat t3.txt
...
15736 pipe2([3, 4], 0) = 0
15736 clone(…) = 15737
15736 clone(…) = 15738
15737 dup2(4, 1) = 1
15738 dup2(3, 0) = 0
15738 execve("/usr/bin/wc", ["wc", "-l"], …) = 0
15737 execve("/usr/bin/grep", ["grep", "ERROR", "logs/system.log"], …) = 0
...
```

Reading the trace

- **pipe2([3, 4]):** the shell gets a pipe: fd 3 = read end, fd 4 = write end.
- **clone × 2:** two children (fork).
- **grep child:** dup2(4, 1) — stdout into the pipe.
- **wc child:** dup2(3, 0) — stdin from the pipe.

```
student@okan-lab:~/lab$ ls -l /proc/self/fd | cat
l-wx------ 1 student student 64 Oct 12 08:48 1 -> pipe:[29942]
# ls's fd 1 is not the terminal any more: it is a pipe (number 29942 in the kernel)
```

LINK TO WEEK 5

A Pipe Is a Bounded Buffer Managed by the Kernel



pipe buffer: 64 KiB on Linux (16 pages × 4 KiB)

- **Buffer full:** write() blocks — the writer waits.
- **Buffer empty:** read() blocks — the reader waits.
- **Writer closed and buffer empty:** read() returns 0 = end of file.

Week 5

The producer–consumer problem — solved for you by the kernel.

```
student@okan-lab:~/lab$ time (sleep 2 | sleep 2)
real    0m2.005s
user    0m0.003s
sys     0m0.002s
student@okan-lab:~/lab$ yes | head -n 3
y
y
y
```

PART 3

Filters: grep, sort, uniq, cut, wc, tr

Each reads lines from files or stdin and writes lines to stdout.

grep: Print the Lines That Match a Pattern

```
student@okan-lab:~/lab$ grep ERROR logs/system.log | head -n 3
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:56:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:07:00 ERROR failed to mount /dev/sdb1
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep -i "TALK" docs/linux_quotes.txt
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -n "backup" logs/system.log | head -n 4
6:2026-10-12 09:55:00 INFO  backup finished
7:2026-10-12 09:06:00 INFO  backup started
10:2026-10-12 09:39:00 INFO  backup finished
29:2026-10-12 10:08:00 INFO  backup started
student@okan-lab:~/lab$ grep -v INFO logs/system.log | head -n 4
2026-10-12 09:11:00 WARN  high memory usage
2026-10-12 09:22:00 WARN  high memory usage
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:23:00 WARN  temperature 71C
student@okan-lab:~/lab$ grep -vc INFO logs/system.log
41
```

Option	Meaning
`-i`	ignore upper/lower case
`-n`	show line numbers
`-v`	invert: lines NOT matching
`-c`	count matching lines
`-w`	match whole words only
`-r`	search directories recursively

Form: grep [options] PATTERN [files]. Quote the pattern if it has spaces or special characters.

Whole Words (-w) and Whole Directories (-r)

```
student@okan-lab:~/lab$ grep "Week 1" docs/syllabus.txt
Week 1  Introduction and history
Week 10 File and directory operations
Week 11 Permissions, ownership and users
Week 12 Text processing and redirection
Week 13 Processes and job control
Week 14 Introduction to shell scripting
Week 15 Final exam
student@okan-lab:~/lab$ grep -w "Week 1" docs/syllabus.txt
Week 1  Introduction and history
student@okan-lab:~/lab$ grep -r "Project" projects
projects/beta/README.md:# Project beta
projects/alpha/README.md:# Project alpha
```

-w

"Week 1" also matches inside "Week 10" … "Week 15". With -w the match must be a whole word: not followed (or preceded) by a letter, digit or _.

-r

Searches every file below a directory and prefixes each line with its file name. The order is the order in the directory, not alphabetical.

Simple Regular Expressions

Symbol	Matches	Example
`^`	start of the line	`^Week`
`$`	end of the line	`s$`
`.`	any ONE character	`c.de → code`
`[ ]`	one character from a set	`[0-9]  [aeiou]`
`*`	the item before it, 0 or more times	`[0-9]*`
`\.`	a literal dot	`5\.38`

Always put the pattern in quotes: otherwise the shell may expand * and [ ] as file-name wildcards (Week 10) before grep sees them.

```
student@okan-lab:~/lab$ grep "^Week [0-9] " docs/syllabus.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
...
student@okan-lab:~/lab$ grep "s$" docs/syllabus.txt
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
Week 4  Threads and CPU scheduling basics
Week 7  Virtual memory and file systems
Week 10 File and directory operations
Week 11 Permissions, ownership and users
student@okan-lab:~/lab$ grep "c.de" docs/linux_quotes.txt
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -c "5.38" logs/access.log
39
student@okan-lab:~/lab$ grep -c "5\.38" logs/access.log
37
student@okan-lab:~/lab$ grep ",ECT,[0-9]*,9[0-9]$" data/students.csv
2026001,Elif Yilmaz,ECT,96,99
2026015,Deniz Celik,ECT,95,96
```

sort: Alphabetical, Numeric, by Column

```
student@okan-lab:~/lab$ sort data/cities.txt | head -n 4
Ankara
Ankara
Antalya
Antalya
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort | head -n 3
100
35
36
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort -n | head -n
3
35
36
36
student@okan-lab:~/lab$ sort -t, -k5 -nr data/students.csv | head -n 3
2026010,Onur Dogan,ME,73,100
2026001,Elif Yilmaz,ECT,96,99
2026020,Emre Dogan,ME,64,98
```

Options

- **-n** numeric: 35 < 100 (plain sort compares characters: "100" < "35" because '1' < '3').
- **-r** reverse (largest first).
- **-t,** the field separator is a comma.
- **-k5** sort by field 5 (here: final grade).
- **-u** unique: drop repeated lines.

uniq: Collapse Adjacent Repeats — So Sort First!

```
student@okan-lab:~/lab$ uniq -c data/cities.txt | head -n 4
      1 Antalya
      1 Trabzon
      1 Konya
      1 Bursa
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c
      2 Ankara
      6 Antalya
      5 Bursa
      5 Eskisehir
      1 Istanbul
      2 Izmir
      4 Konya
      5 Trabzon
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c | sort
-rn | head -n 3
      6 Antalya
      5 Trabzon
      5 Eskisehir
```

```
student@okan-lab:~/lab$ cut -d, -f2 data/students.csv | sort |
uniq -d
Elif Yilmaz
Mehmet Aydin
Onur Dogan
Selin Yildiz
```

Remember

- uniq only compares a line with the one just before it.
- **-c** count each group;
- **-d** print only repeated lines (here: names that appear twice).
- **sort | uniq -c | sort -rn** = "how many of each, most common first" — the most useful pipeline of the day.

cut Takes Columns, wc Counts

```
student@okan-lab:~/lab$ head -n 3 data/students.csv | cut -d, -f2,5
name,final
Elif Yilmaz,99
Zeynep Dogan,36
student@okan-lab:~/lab$ head -n 2 logs/access.log | cut -d" " -f1,7,9
10.0.1.10 /about.html 404
10.0.1.10 /api/grades 200
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -c12-16
09:00
09:11
09:22
student@okan-lab:~/lab$ wc logs/system.log
 120  669 5207 logs/system.log
student@okan-lab:~/lab$ wc -l data/*
  30 data/cities.txt
  20 data/numbers.txt
  41 data/students.csv
  91 total
```

cut

- **-d',' -f2,5** fields 2 and 5, separated by commas.
- **-d" " -f7** field 7, separated by single spaces.
- **-c12-16** characters 12 to 16 (fixed columns).

wc

Prints lines, words, bytes: 120 lines, 669 words, 5207 bytes. -l, -w, -c for just one of them.

tr: Translate, Delete and Squeeze Characters

```
student@okan-lab:~/lab$ head -n 2 docs/linux_quotes.txt | tr a-z A-Z
TALK IS CHEAP. SHOW ME THE CODE.
GIVEN ENOUGH EYEBALLS, ALL BUGS ARE SHALLOW.
student@okan-lab:~/lab$ head -n 2 data/students.csv | tr "," ";"
id;name;department;midterm;final
2026001;Elif Yilmaz;ECT;96;99
student@okan-lab:~/lab$ echo "Hello, World!" | tr -d ",!"
Hello World
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -d" " -f4



student@okan-lab:~/lab$ head -n 3 logs/system.log | tr -s " " | cut -d" " -f4-
network link up
high memory usage
high memory usage
```

Three jobs

- **tr a-z A-Z** replace each character of set 1 by the one of set 2.
- **tr -d set** delete those characters.
- **tr -s " "** squeeze repeats into one.

Why the empty lines?

"INFO network" has TWO spaces, so cut sees an empty field 4. tr -s " " squeezes them first; -f4- means field 4 to the end.

CHEAT SHEET

Redirection and Filters at a Glance

You want to...	Use	Example from today
save / append output	> f >> f	ls data > list.txt
read input from a file	< f	wc -l < logs/access.log
save or hide errors	2> f 2> /dev/null	find . -name "*.md" 2> /dev/null
everything into one file	> f 2>&1 &> f	ls data nosuch > all.txt 2>&1
connect two programs	cmd1 | cmd2	grep ERROR logs/system.log | wc -l
find matching lines	grep -i -n -v -c -w -r	grep -c " 404 " logs/access.log
sort lines	sort -n -r -t, -k	sort -t, -k5 -nr data/students.csv
count each value	sort | uniq -c	sort data/cities.txt | uniq -c
take columns	cut -d -f / -c	cut -d, -f2,5 data/students.csv
change characters	tr tr -d tr -s	tr a-z A-Z < docs/linux_quotes.txt

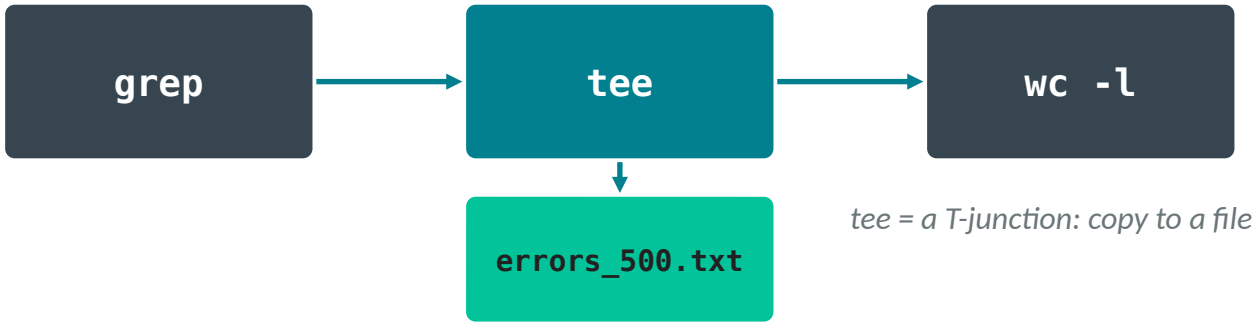
INTERMISSION

10-Minute Break

Next: head, tail, tee, find, xargs — then real questions answered with one-line pipelines.

head, tail and tee in Pipelines

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | head -n 2
2026001,Elif Yilmaz,ECT,96,99
2026002,Zeynep Dogan,ME,95,36
student@okan-lab:~/lab$ head -n 120 logs/access.log | tail -n 1
10.0.3.24 - - [12/Oct/2026:12:53:47 +0300] "GET /index.html HTTP/1.1" 404 4270
student@okan-lab:~/lab$ grep " 500 " logs/access.log | tee errors_500.txt | wc -l
24
student@okan-lab:~/lab$ wc -l errors_500.txt
24 errors_500.txt
```



tee = a T-junction: copy to a file AND pass on

Useful forms

- **head -n 5** first 5 lines.
- **tail -n 5** last 5 lines.
- **tail -n +2** from line 2 on: skips a header.
- **head -n 120 | tail -n 1** exactly line 120.
- **tail -f log** follow a growing file (Ctrl+C to stop).
- **tee -a f** append instead of overwrite.

find: Search the Directory Tree by Name, Type, Size, Age

```
student@okan-lab:~/lab$ find docs -name "*.txt"
docs/notes_02.txt
docs/syllabus.txt
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab$ find . -type d -name src
./src
./projects/beta/src
./projects/alpha/src
student@okan-lab:~/lab$ find . -size +10k
./logs/access.log
student@okan-lab:~/lab$ find photos -iname "*.jp*g" | wc -l
14
student@okan-lab:~/lab$ touch -d "2026-09-01" docs/notes_01.txt
student@okan-lab:~/lab$ find docs -mtime +30
docs/notes_01.txt
```

Test	Meaning
`-name "*.txt"`	name matches (wildcards in quotes)
`-iname`	same, ignoring case
`-type f / d`	regular file / directory
`-size +10k`	bigger than 10 KiB (-10k: smaller)
`-mtime +30`	modified more than 30 days ago
`-mtime -1`	modified in the last 24 hours

Form: find WHERE TESTS. Several tests = all must hold (AND). Unlike ls, find walks the whole tree below WHERE.

Acting on What find Finds: -exec and xargs

```
student@okan-lab:~/lab$ find . -name "*.c" -exec wc -l {} \;
7 ./src/hello.c
1 ./projects/beta/src/main.c
1 ./projects/alpha/src/main.c
student@okan-lab:~/lab$ find . -name "*.c" | xargs wc -l
  7 ./src/hello.c
  1 ./projects/beta/src/main.c
  1 ./projects/alpha/src/main.c
  9 total
student@okan-lab:~/lab$ find docs -name "*.txt" | xargs wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
wc: docs/final: No such file or directory
wc: report.txt: No such file or directory
 23 total
student@okan-lab:~/lab$ find docs -name "*.txt" -print0 | xargs -0 wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
  2 docs/final report.txt
 25 total
```

Two ways

- **-exec cmd {} \;** runs cmd once per file; {} = the file name.
- **| xargs cmd** turns stdin lines into ARGUMENTS: one wc with all names (note the total).
- **-print0 | xargs -0** safe for names with spaces.

Safety

Before find … -exec rm {} \; or … | xargs rm, run the same line WITHOUT rm and read the list. rm has no undo.

PART 5

Real Data, One-Line Answers

logs/access.log · logs/system.log · data/students.csv

Which Field Is Where?

logs/access.log — cut -d" " -fN

-f1	-f2	-f3	-f4	-f5	-f6	-f7	-f8	-f9	-f10
10.0.1.10	-	-	[12/Oct/2026:08:00:00	+0300]	"GET	/about.html	HTTP/1.1"	404	442
IP			*date and time*	*zone*	*method*	*page*	*protocol*	*status*	*bytes*

logs/system.log — fixed columns: cut -cA-B

-c1-10	-c12-19	-c21-25	-c27-
2026-10-12	09:11:00	WARN	high memory usage
date	*time (hour: 12-13)*	*level*	*message*

data/students.csv — cut -d, -fN (line 1 is a header!)

-f1	-f2	-f3	-f4	-f5
2026001	Elif Yilmaz	ECT	96	99
id	*name*	*department*	*midterm*	*final*

Before any pipeline

- head -n 3 the file.
- Find the separator: space, comma, fixed columns?
- Count the fields from 1.

METHOD

Build a Pipeline One Stage at a Time

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | head -n 3
404
200
500
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | head -n 3
200
200
200
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
```

1

Extract

cut the column you need; check with head.

2

Group

sort brings equal values together.

3

Count

uniq -c counts each group.

4

Rank

sort -rn: largest count first; head -n N for a top N.

Top Pages, Top Visitors, Broken Links

```
student@okan-lab:~/lab$ cut -d" " -f7 logs/access.log | sort | uniq -c | sort -rn | head -n 5
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
     33 /images/logo.png
     32 /api/grades
student@okan-lab:~/lab$ cut -d" " -f1 logs/access.log | sort | uniq -c | sort -rn | head -n 3
     58 10.0.2.17
     56 10.0.4.31
     52 10.0.6.45
student@okan-lab:~/lab$ grep " 404 " logs/access.log | cut -d" " -f7 | sort | uniq -c | sort -rn | head -n 3
      9 /images/logo.png
      7 /index.html
      7 /about.html
```

Errors per Hour, Warnings, Departments, Best Finals

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep ERROR logs/system.log | cut -c12-13 | uniq -c
      3 09
      4 10
      1 12
      3 13
      2 14
student@okan-lab:~/lab$ grep WARN logs/system.log | cut -c27- | sort | uniq -c
     11 high memory usage
     17 temperature 71C
```

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f3 | sort | uniq -c | sort -rn
     17 ECT
     11 ME
     10 CS
      2 EE
student@okan-lab:~/lab$ tail -n +2 data/students.csv | sort -t, -k5 -nr | head -n 3 | cut -d, -f2,3,5
Onur Dogan,ME,100
Elif Yilmaz,ECT,99
Emre Dogan,ME,98
```

Lab: Redirection and grep (Parts A and B)

A1 Save the file list of data/ into list.txt, then append the list of docs/. How many lines?

A2 Run ls data nosuch so that the listing goes to ok.txt and the error to err.txt.

A3 Print only the number of lines of logs/system.log — without the file name.

A5 find . -name "*.md" complains about a folder you may not read. Hide only the errors.

B2 Show the first 3 lines of system.log that are NOT INFO, with line numbers.

B3 How many requests in access.log got the status 404?

B4 Which students belong to the EE department?

B5 How many requests came from 10.0.5.38? (Careful with the dots!)

Lab: One-Line Pipelines (Parts C and D)

C1 How many students are in each department?

C2 Names and final grades of the top 3 students by final grade.

C4 The 3 most requested pages of the web site.

C5 Which page produced the most 404 errors?

C6 How many ERROR lines does system.log contain per hour?

D1 Which student names appear more than once in students.csv?

D2 Save all status-500 lines into errors_500.txt AND print how many there are — one line.

D4 How many different IP addresses visited the site? Which one made the fewest requests?

Six Mistakes Everybody Makes Once

```
student@okan-lab:~/lab$ cp data/cities.txt c.txt
student@okan-lab:~/lab$ sort c.txt > c.txt
student@okan-lab:~/lab$ wc -l c.txt
0 c.txt
student@okan-lab:~/lab$ grep user login logs/system.log
grep: login: No such file or directory
logs/system.log:2026-10-12 09:44:00 INFO  user login
...
student@okan-lab:~/lab$ ls data > wc
student@okan-lab:~/lab$ ls
README.txt  c.txt  docs  photos    scripts  wc
backup      data   logs  projects  src
student@okan-lab:~/lab$ sort -t, -k5 -n data/students.csv |
head -n 2
id,name,department,midterm,final
2026032,Onur Dogan,ECT,41,35
```

sort f > f

The shell empties f BEFORE sort reads it: 0 lines. Write to a new file, then mv.

grep user login f

Unquoted: grep searches "user" in a file named login. Use "user login".

ls data > wc

> is not |. This creates a file named wc. Use ls data | wc -l.

Header line

"id,name,..." sorts as 0 with -n. Skip it: tail -n +2.

uniq without sort

Counts only adjacent repeats. Use sort | uniq -c.

2>&1 > f

Order is left to right: the error stays on screen. Write > f 2>&1.

QUICK QUIZ

Check Your Understanding

1. What is the difference between `cmd > out.txt 2>&1` and `cmd 2>&1 > out.txt` ?

2. Why does `uniq -c data/cities.txt` report "1 Antalya" although Antalya appears 6 times? Fix it.

3. Write a pipeline that prints the 3 IP addresses with the most requests in logs/access.log.

4. Which system calls does the shell use to run `grep ERROR logs/system.log | wc -l` ?

Week 12 — Key Takeaways

- **Standard streams:** every process starts with fd 0 (stdin), 1 (stdout), 2 (stderr) — the same write(1, …) we traced in Weeks 2–3.
- **Redirection:** > >> < 2> 2>&1 &> /dev/null. The shell opens the file and uses dup2() between fork() and exec(); the program never knows.
- **Order:** redirections are applied left to right: > f 2>&1 captures both streams, 2>&1 > f does not.
- **Pipes:** cmd1 | cmd2 runs both processes concurrently, linked by a kernel buffer (a bounded buffer, Week 5).
- **Filters:** grep (-i -n -v -c -w -r, ^ $. [ ] *), sort (-n -r -t -k), uniq (-c -d, after sort), cut, wc, tr, head, tail, tee.
- **find + xargs:** search the tree by name, type, size, age; -print0 | xargs -0 for names with spaces.
- **Method:** extract → sort → uniq -c → sort -rn → head, one stage at a time.

NEXT WEEK

Processes & Job Control (Week 13)

Week 13 will cover:

- ps, pstree, top: watching processes, PIDs and states (Week 3)
- Foreground and background: &, jobs, fg, bg, Ctrl+Z, Ctrl+C
- Signals: kill, SIGTERM vs SIGKILL — and why yes | head stops
- nice and renice: process priority (Week 5)
- A zombie process you can see with your own eyes

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz